

# async

Asynchronous programming in Python allows a program to handle multiple tasks concurrently without blocking the execution of other code.

Unlike the standard synchronous code, which waits for one task to finish before starting the next, asynchronous code "pause" during the long-running I/O operations (like network requests or database queries) to let other tasks run in the meantime.

`async` = used to define a coroutine, a special type of function that can be paused and resumed

`await` = used inside an `async` function to pause execution until a specific task (like a web request) is complete.

`async` in Python is used for **asynchronous programming** — a way to handle tasks that spend time waiting, without blocking the whole program.

It's especially useful for:

- network requests
- APIs
- databases
- file downloads
- web servers
- websockets
- MCP clients/servers

## Normal (synchronous) code

Regular Python runs line by line.

**Example:**

```
import time

def task():
    print("Start")
    time.sleep(3)
    print("Done")

task()
print("Next")
```

Output:

```
Start
(wait 3 seconds)
```

Done  
Next

During `sleep(3)` :

- the whole program pauses
- nothing else can happen

This is called **blocking**

## Async code

With async:

```
import asyncio

async def task():
    print("Start")
    await asyncio.sleep(3)
    print("Done")

asyncio.run(task())
```

`await` means:

“Pause this task temporarily and let other tasks run.”

This is **non-blocking**.

## Key async keywords

**async def** Defines an asynchronous function.

```
async def hello():
    print("Hello")
```

This creates a **coroutine**.

It does NOT run immediately.

**await**

Used inside async functions.

```
await something()
```

Means:

“Wait for this async operation to finish.”

But while waiting, Python can run other async tasks.

## `asyncio.run()`

Starts the async event loop.

```
asyncio.run(main())
```

Without this, async functions won't execute.

## Coroutine?

When you call an async function:

```
main()
```

it does NOT run immediately.

It returns a coroutine object:

```
<coroutine object main at 0x...>
```

To actually execute it:

```
await main()
```

or

```
asyncio.run(main())
```

## Why async?

Suppose you make 3 API calls.

### Synchronous version

```
download1()  
download2()  
download3()
```

If each takes 3 seconds:

```
3 + 3 + 3 = 9 seconds
```

### Async version

```
await asyncio.gather(  
    download1(),  
    download2(),
```

```
download3()  
)
```

All run concurrently.

Total time:

~3 seconds

--> Huge speed improvement for I/O tasks.

## Event loop

Asyncio uses an **event loop**.

Think of it as a manager:

```
Task A waiting for network  
↓  
Run Task B  
  
Task B waiting  
↓  
Run Task C
```

Instead of sitting idle during waiting time, Python switches tasks.

## **await** only works in async functions

This is INVALID:

```
def test():  
    await something()
```

You must use:

```
async def test():  
    await something()
```

## Async ≠ parallel CPU processing

Important distinction:

**Async is best for:**

- waiting
- networking
- I/O

**Not best for:**

- heavy math

- CPU-intensive work
- video rendering

**For CPU-heavy tasks, use:**

- multiprocessing
- threads
- distributed systems

## Common async flow

Typical structure:

```
import asyncio

async def work():
    print("Working...")
    await asyncio.sleep(1)
    print("Done")

async def main():
    await work()

asyncio.run(main())
```

## Most important rule

Async only helps when operations spend time waiting.

**Examples:**

- HTTP requests
- databases
- websocket communication
- SSE streams
- MCP tools
- file I/O